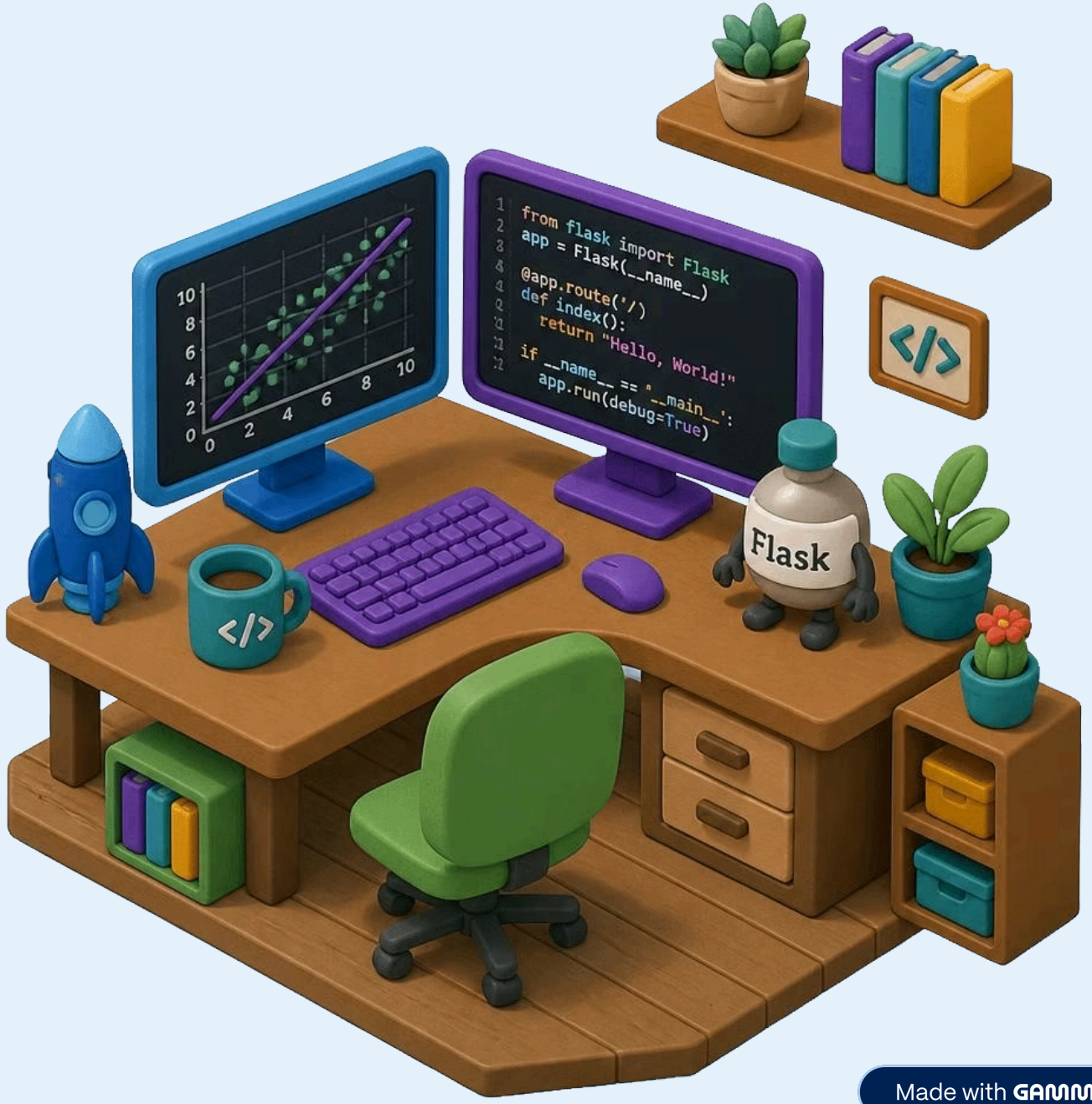
AULA FINAL

Flask + clustering na prática

Construa um app web em Python que recebe dados, calcula clusters com K-Means e devolve o gráfico interativo no navegador.

Mentor Prof. Cloves Rocha



Python · Flask · Scikit-Learn · Pandas · Matplotlib

Made with GAMMA

O DESAFIO

Um app web que agrupa dados brutos

Ao final da aula, um app no navegador: o usuário envia dados e recebe grupos com K-Means.

Entrada do usuário

Upload de arquivo CSV com colunas numéricas e definição do valor de `k`.

Processamento no servidor

O Flask invoca o K-Means no Scikit-Learn e gera a visualização gráfica dos dados.

Resultado visual

Página web renderizada com métricas de coesão e gráfico de dispersão colorido.



Made with GAMMA

CONCEITO

Clustering agrupa sem rótulos prévios

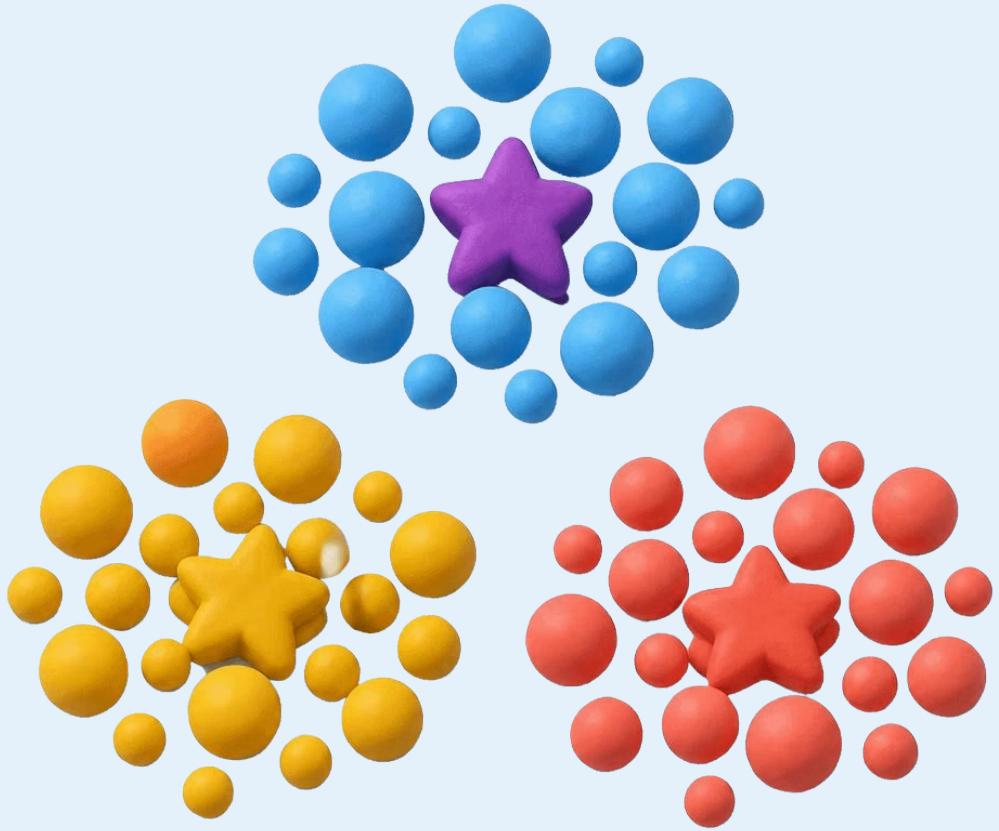
Diferente da classificação supervisionada, aqui não temos gabarito nem respostas certas. O modelo identifica padrões intrínsecos e vizinhanças nos dados.

Segmentação de clientes

Agrupar por comportamento de compra, frequência e ticket médio para campanhas direcionadas.

Agrupamento de notícias

Detectar temas emergentes reunindo artigos semanticamente próximos sem pré-definir categorias.



Sem rótulos: a distância euclidiana define quem pertence ao mesmo grupo.

Made with GAMMA

ALGORITMO

K-Means em quatro passos

Um loop iterativo de convergência baseado na minimização da distância quadrática até o centroide.



1. Inicializar

Escolher k centroides iniciais no espaço de atributos.



2. Atribuir

Associar cada amostra ao centroide geometricamente mais próximo.



3. Recalcular

Reposicionar os centroides pela média dos pontos atribuídos.



4. Convergir

Repetir até que os centros não mudem de posição.

Made with GAMMA

SETUP

Preparando o ambiente e estrutura

Isole o interpretador em um ambiente virtual e instale a pilha básica de processamento numérico e servidor.

Comandos no terminal

Criação do ambiente virtual e instalação das bibliotecas essenciais:

```
python -m venv venv
source venv/bin/activate  # Windows:
venv\Scripts\activate
pip install flask scikit-learn pandas matplotlib
```

Estrutura do projeto

Organização dos diretórios esperada pelo Flask:

```
projeto-kmeans/
├── app.py
├── templates/
│   └── index.html
└── static/
    └── cluster_plot.png
```

`templates/ guarda HTMLs; static/ armazena imagens geradas.`

Made with GAMMA

ROTA INICIAL

Primeira rota Flask no ar

Configure a instância da aplicação e monte a rota padrão que renderiza a interface base.

app.py — Servidor mínimo

```
from flask import Flask, render_template
app = Flask(__name__)

@app.route("/")
def index():
    return render_template("index.html")

if __name__ == "__main__":
    app.run(debug=True)
```

1

Salvar o código

Grave o conteúdo no arquivo `app.py` na raiz do projeto.

2

Subir o servidor

Execute `flask run` no terminal com venv ativo.

3

Verificar no browser

Abra `http://127.0.0.1:5000` e veja a página responder.

Made with GAMMA

INTERFACE

Formulário: upload do CSV e escolha do k

O formulário HTML envia o arquivo e o campo numérico; a rota POST lê o CSV com Pandas.

index.html — Formulário

- Arquivo CSV: campo `arquivo` obrigatório.
- Quantidade de clusters (k): campo numérico de 2 a 10, com valor inicial 3.
- Botão: **Agrupar dados**.
- Envio POST com `enctype="multipart/form-data"`, obrigatório no upload.

app.py — Leitura do POST

- A rota `/` aceita GET e POST.
- No POST, `request.files["arquivo"]` recebe o arquivo; `request.form.get("k", 3)` obtém k.
- `pd.read_csv(arquivo)` lê os dados para o clustering.
- `render_template("index.html")` devolve a página.

Made with GAMMA

PROCESSAMENTO

Rodando o K-Means no servidor

Treinamos nas duas primeiras colunas e devolvemos o gráfico no template.

1. Ajustar e agrupar

```
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

X = df.iloc[:, [0, 1]].values
kmeans = KMeans(n_clusters=k,
  random_state=42)
labels = kmeans.fit_predict(X)
```

O scikit-learn atribui um grupo a cada ponto.

matplotlib.use("Agg") evita erros de interface gráfica em servidores web.

2. Gerar e devolver

```
plt.figure(figsize=(6, 4))
plt.scatter(*X.T, c=labels,
  cmap="viridis", alpha=0.7)
plt.scatter(*kmeans.cluster_centers_.T,
  s=180, c="red", marker="X",
  label="Centróides")
plt.savefig("static/cluster_plot.png",
  bbox_inches="tight")
plt.close()
```

Centros em vermelho; a imagem em `static/` é carregada pelo template HTML.

Made with GAMMA

MÉTRICAS

Avaliando e escolhendo o k ideal

A inércia mede a distância interna dos pontos ao seu centróide. O ponto de inflexão indica o número ótimo de grupos.



Método do Cotovelo

O ganho diminui após **k = 3**, revelando o cotovelo natural do conjunto.

Coeficiente Silhouette

Avalia coesão interna vs separação externa. Varia de -1 a +1; quanto mais próximo de 1, melhor delimitados estão os grupos.

No app, mostre a métrica ao lado do gráfico.

Made with GAMMA



ENTREGA

Deploy e próximos passos

Nosso app Flask está funcional! Para consolidar e expandir seu portfólio:

Publicação do código

Suba no GitHub com `requirements.txt` e faça deploy gratuito (Render ou Railway).

Novos algoritmos

Explore novos datasets com DBSCAN para formas irregulares e Clustering Hierárquico com dendrogramas.

Pré-processamento

Adicione `StandardScaler` para variáveis com escalas diferentes antes do fit.

Parabéns pelo projeto concluído!

Made with GAMMA